

Requirements for Numerical P Systems with Boolean Condition (BNPS) Simulator Development

1. System Components

The tool shall consist of three (3) core executable/configuration files:

File Name	Purpose
bnps.py	Main Python-based entry point for serial and parallel execution control.
bnps.cu	CUDA-based parallel kernel implementation for BNPS simulation.
check.pep	Sample input file for validating BNPS simulator functionality.

2. Execution Commands

The tool shall support the following command-line invocations:

Command	Execution Mode	Description
python3 bnps.py check.pep -n <stepcount>	Serial	Executes the BNPS simulation using a pure Python serial implementation.
python3 bnps.py check.pep -p <stepcount>	Parallel	Executes the BNPS simulation using CUDA parallel implementation (bnps.cu).

3. Input Specifications

The CUDA parallel implementation shall require an intermediate input file with the following characteristics:

- File Name: **input.txt**
- **Generation:** Created automatically by **bnps.py** when parallel execution (**-p** flag) is invoked.
- **Content Structure:** Shall contain the complete BNPS model description in a parsable format, including:
 - Membrane structures and hierarchy
 - Production rules with associated thresholds
 - Repartition (redistribution) protocols
 - Variable identifiers and their initial values

4. Functional Requirements

4.1 Serial Implementation (**-n** flag)

- Shall execute BNPS simulation sequentially in Python.
- Shall parse the **.pep** input file directly.
- Shall produce deterministic results consistent with BNPS theoretical semantics.

4.2 Parallel Implementation (**-p** flag)

- Shall invoke the CUDA kernel (**bnps.cu**) through the Python entry point (**bnps.py**).
- Shall generate **input.txt** from the provided **.pep** file before CUDA execution.
- The CUDA kernel shall:
 - Parse **input.txt** to extract BNPS model data.
 - Distribute computation across GPU threads as applicable.
 - Adhere to BNPS semantics while leveraging parallel processing.

5. Interface Requirements

- The command-line interface shall follow the format:

```
python3 bnps.py <input_file> [ -n | -p ] <stepcount>
```

- Flags **-n** and **-p** shall be mutually exclusive.
- Clear error messages shall be displayed for:
 - Missing or malformed input files.
 - Incorrect flag usage.
 - CUDA runtime errors (where applicable).

6. Validation Requirement

- The tool shall be verifiable using the provided **check.pep** sample file.
- Both serial and parallel execution shall produce identical results for the same input model.

7. Non-Functional Requirements

- **Portability:** The Python components shall run on any system with Python 3.x.
- **CUDA Dependency:** The parallel implementation requires a CUDA-compatible GPU and toolkit.
- **Performance:** The parallel implementation shall demonstrate speedup over serial execution for large-scale BNPS models.

BNPS.pep

```
bnps = {  
  
  H = {1,2,3};  
  structure = [ 1 [ 2 ] 2 [ 3 ] 3 ] 1;  
  
  # First membrane with simple programs  
  1 = {  
    var = { a, b, c };  
  
    # Programs  
    pr = { a + 3*b | a = 1  $\wedge$  b > 0  $\rightarrow$  2 | c + 1 | a + 1 | b };  
    pr = { b |  $\neg$ (b = 0)  $\rightarrow$  1 | d };  
    pr = { a | a = 1  $\oplus$  b = 1  $\rightarrow$  1 | e };  
  
    # Initial values  
    var0 = (1, 1, 1);  
  
    # Second membrane with simple programs  
    2 = {  
      var = { d };  
  
      # Programs  
      pr = { -2*d |  $\neg$ (d > 0)  $\rightarrow$  1 | a + 1 | b };  
  
      # Initial values  
      var0 = (0);  
    };  
  
    # Third membrane with simple programs  
    3 = {  
      var = { e };  
  
      # Programs  
      pr = { -2*e |  $\neg$ (e > 0)  $\oplus$  e = -1  $\rightarrow$  1 | a + 1 | b };  
  
      # Initial values  
      var0 = (0);  
    };  
  };  
}
```

Example:

Membrane Structure: $[[]_2 []_3]_1$ (membranes 2 and 3 inside membrane 1)

Initial Values:

Membrane 1: $a=1, b=1, c=1$

Membrane 2: $d=0$

Membrane 3: $e=0$

Programs:

1. $pr_{1,1}: a + 3b \mid a = 1 \wedge b > 0 \rightarrow 2 \mid c + 1 \mid a + 1 \mid b$

2. $pr_{2,1}: b \mid \neg(b = 0) \rightarrow 1 \mid d$

3. $pr_{3,1}: a \mid a = 1 \oplus b = 1 \rightarrow 1 \mid e$

4. $pr_{1,2}: -2*d \mid \neg(d > 0) \rightarrow 1 \mid a + 1 \mid b$

5. $pr_{1,3}: -2*e \mid \neg(e > 0) \oplus e = -1 \rightarrow 1 \mid a + 1 \mid b$

Working Mode: All-parallel

Initial State (t=0)

Membrane 1: $a=1, b=1, c=1$

Membrane 2: $d=0$

Membrane 3: $e=0$

Phase 1: Determine Boolean Conditions**Membrane 1 programs:**

$a + 3*b \mid a = 1 \wedge b > 0 \rightarrow 2 \mid c + 1 \mid a + 1 \mid b$

Condition: $a = 1 \wedge b > 0$

Check: $a=1$ (true) $\wedge b=1>0$ (true) $\rightarrow$ **TRUE**

$b \mid \neg(b = 0) \rightarrow 1 \mid d$

Condition: $\neg(b = 0)$

Check: $b=0$? No (false), so $\neg(\text{false}) =$ **TRUE**

$a \mid a = 1 \oplus b = 1 \rightarrow 1 \mid e$

Condition: $a = 1 \oplus b = 1$ (exclusive OR)

Check: $a=1$ (true) $\oplus b=1$ (true) $\rightarrow \text{true} \oplus \text{true} =$ **FALSE**

So programs 1 and 2 in membrane 1 will execute, program 3 will not.

Membrane 2 program:

$-2*d \mid \neg(d > 0) \rightarrow 1 \mid a + 1 \mid b$

Condition: $\neg(d > 0)$

Check: $d=0 > 0$? false, so $\neg(\text{false}) = \text{TRUE}$

Membrane 3 program:

$-2*e \mid \neg(e > 0) \oplus e = -1 \rightarrow 1 \mid a + 1 \mid b$

Condition: $\neg(e > 0) \oplus e = -1$

Check: $e=0 > 0$? false, so $\neg(\text{false}) = \text{true}$

$e = -1$? false

$\text{true} \oplus \text{false} = \text{TRUE}$

Phase 2: Compute Production Functions**Membrane 1, Program 1: $a + 3*b$**

Use current values: $a=1, b=1, c=1$

Compute: $1 + 3 \times 1 = 4$

After computation, variables a and b are consumed (set to 0)

Membrane 1, Program 2: b

Use current values: $a=0$ (already consumed), $b=0$ (already consumed), $c=1$

Wait, b was already consumed! $b=0$ now

Compute: 0 (since $b=0$ after consumption)

Note: This is a subtle point - since program 1 consumed b, program 2 sees $b=0$

Membrane 2, Program 1: $-2*d$

Current: $d=0$

Compute: $-2 \times 0 = 0$

Membrane 3, Program 1: $-2*e$

Current: $e=0$

Compute: $-2 \times 0 = 0$

Phase 3: Perform Repartition Protocol

Parameter	Membrane 1 – Program 1	Membrane 1 – Program 2	Membrane 2 – Program 1	Membrane 3 – Program 1
Production value	4	0	0	0
Repartition rule	$2 c + 1 a + 1 b$	$1 d$	$1 a + 1 b$	$1 a + 1 b$
Variables involved	c, a, b	d	a, b	a, b
Variable location	Membrane 1	Membrane 2 (child)	Membrane 1 (parent)	Membrane 1 (parent)
Distribution coefficients	c:2, a:1, b:1	d:1	a:1, b:1	a:1, b:1
Sum of coefficients (C)	4	1	2	2
Distribution rate (q)	$4 / 4 = 1$	$0 / 1 = 0$	$0 / 2 = 0$	$0 / 2 = 0$
Distributed value(s)	c=2, a=1, b=1	d=0	a=0, b=0	a=0, b=0

Final Variable Updates (t=1)

Membrane 1:

a:

Appeared in production? Yes (program 1 consumed it)

Appears in repartition? Yes (programs 1, 2, 3 all send to a)

From program 1: 1

From program 2 (membrane 2): 0

From program 3 (membrane 3): 0

Total: 1

b:

Appeared in production? Yes (program 1 consumed it)

Appears in repartition? Yes (programs 1, 2, 3 all send to b)

From program 1: 1

From program 2 (membrane 2): 0

From program 3 (membrane 3): 0

Total: 1

c:

Appeared in production? No

Appears in repartition? Yes (program 1 sends to c)

Previous value: 1

From program 1: 2

Total: $1 + 2 = 3$

Membrane 2:

d:

Appeared in production? Yes (program 1 consumed d)

Appears in repartition? Yes (program 2 from membrane 1 sends to d)

From program 2 (membrane 1): 0

Total: 0

Membrane 3:

e:

Appeared in production? Yes (program 1 consumed e)

Appears in repartition? No

According to rule (i): reset to 0

Total: 0

Final State (t=1)

Membrane 1: $a=1$, $b=1$, $c=3$

Membrane 2: $d=0$

Membrane 3: $e=0$